

LLaMA 2 Fine Tuning

Building Your Own LLaMA, Step by Step

Guide Categories



What Is LLaMA 2?

LLaMA2, introduced by Meta in 2023, is an open source large language model (LLMs). It is a part of the LLaMA (Language Large Model) family, which encompasses a range of models with varying capacities, from 7 billion to 70 billion parameters.

The number of parameters is a key aspect of LLMs, determining their capacity to learn from data and generate responses. The greater the number of parameters, the more nuanced and complex the model's capabilities generally are. The LLaMA series of models is unique in that it provides a range of model variants, each with a different number of parameters, for different use cases.

LLaMA2 has been trained on an extensive dataset of 2 trillion tokens, offering a context length of 4,096 tokens, double that of its predecessor, LLaMA1. Context length refers to the amount of input text the model can consider at one time, which is crucial for understanding and generating coherent and contextually relevant responses.

LLaMA2 also features models specifically fine-tuned for certain applications. For example, LLaMA Chat, optimized for dialogue use cases, has been trained on over 1 million human annotations to enhance its conversational abilities. Another variant, Code LLaMA, focuses on code generation, supporting multiple programming languages like Python, Java, and C++. It's trained on a massive corpus of 500 billion tokens of code, showcasing its specialization in programming-related tasks.

This is part of a series of articles about generative AI.

In this article:

- [Key Concepts in LLM Fine Tuning](#)
 - [Supervised Fine-Tuning \(SFT\)](#)

We use cookies on our site to give you the best experience possible. By continuing to browse the site, you agree to this use. For more information on how we use cookies, see our [Privacy Policy](#).

Accept

- [Parameter-Efficient Fine-Tuning \(PEFT\) with LoRA or QLoRA](#)
- [How to Fine-Tune LLaMA 2: Step by Step](#)
- [Optimizing Your AI Infrastructure with Run:ai](#)

Key Concepts in LLM Fine Tuning

Fine-tuning large language models involves adapting the pre-trained model to perform specific tasks or understand particular domains better. This is achieved by training the model on a new dataset that is more focused on the desired task or domain. The fine-tuning process adjusts the weights of the model's neural network, enabling it to make better predictions or generate more accurate responses based on the new data.

Here are a few key concepts commonly used in LLM fine tuning:

Supervised Fine-Tuning (SFT)

Supervised Fine-Tuning (SFT) is a process where a pre-trained language model is further trained (fine-tuned) on a smaller, task-specific dataset under human supervision. The goal is to adapt the general knowledge of the model to specific tasks or domains. For instance, if LLaMA2 needs to be specialized for medical data analysis, it would undergo SFT on a dataset comprising medical texts, patient records, and related literature.

During SFT, the model learns from labeled examples. Each example in the training dataset contains an input (such as a question or a statement) and the corresponding output (like an answer or a continuation of the statement). This method contrasts with unsupervised learning, where the model learns from data without explicit labels. SFT helps the model understand and generate more accurate and relevant responses in specific domains or tasks, thereby enhancing its applicability.

To implement SFT, one typically adjusts the learning rate, batch size, and the number of training epochs. These parameters are crucial for ensuring that the model does not overfit on the specific dataset, which could reduce its performance on more general tasks. Furthermore, evaluation metrics, such as accuracy or F1 score, are used to gauge the model's proficiency on the specific task post-fine-tuning.

Reinforcement Learning from Human Feedback (RLHF)

Reinforcement Learning from Human Feedback (RLHF) is an advanced fine-tuning technique used to further refine the performance of language models like LLaMA2. It involves training the model using feedback derived from human interactions. The process is based on the reinforcement learning paradigm, where the model is encouraged to make decisions that lead to positive outcomes or feedback.

In RLHF, human evaluators interact with the model by providing inputs and then rating or correcting the outputs generated by the model. This feedback serves as a reward signal that guides the model to learn which types of responses are preferred or more accurate in given contexts. The model's objective is to maximize the positive feedback it receives, effectively aligning its responses more closely with human expectations and preferences.

This fine-tuning method is particularly useful for improving the model's performance in complex, subjective tasks such as conversation generation, ethical reasoning, or creative writing. RLHF helps the model to understand nuances and subtleties in

ate, context-sensitive, and

Prompt Template

A prompt template is used in fine-tuning language models like LLaMA2 to generate specific types of outputs. It involves creating templates or patterns that guide the model in generating responses in a desired format or style. This is particularly useful when the model needs to generate responses that conform to certain standards or formats, such as in structured data entry, creative writing, or when answering specific types of queries.

A prompt template typically consists of a fixed part, which sets the context or the format, and a variable part, where the model fills in the information based on the input. For instance, a template for generating weather forecasts might start with "The weather forecast for [location] on [date] is:", and the model would complete the sentence with the forecast details.

The effectiveness of prompt templates depends on how well they are designed to elicit the desired response from the model. They need to be clear, concise, and relevant to the task at hand. Fine-tuning with prompt templates can significantly increase the model's efficiency in generating specific types of responses and can be combined with other fine-tuning methods for more complex tasks.

Parameter-Efficient Fine-Tuning (PEFT) with LoRA or QLoRA

Parameter-Efficient Fine-Tuning (PEFT) is a technique that allows for the fine-tuning of large language models like LLaMA2 without the need to update all of the model's parameters. This is achieved by focusing on a small subset of the model's parameters, making the fine-tuning process more efficient and less resource-intensive. Two popular methods used in PEFT are LoRA (Low-Rank Adaptation) and QLoRA (Quantized Low-Rank Adaptation).

LoRA focuses on modifying only the weights of certain layers within the model, specifically targeting those that are most impactful for the task. This is done by applying low-rank matrices to transform these weights during the forward pass of the model. The advantage of LoRA is that it maintains the model's original parameter count, making it easier to deploy without extensive modifications.

QLoRA, on the other hand, involves quantizing the parameters of the model, reducing the precision of the weights while maintaining performance. This approach is particularly useful for deploying models on resource-constrained environments, as it significantly reduces the model's memory footprint and computational requirements.

Learn more in our detailed guide to [LoRA fine tuning](#)

How to Fine-Tune LLaMA 2: Step by Step

The following tutorial will take you through the steps required to fine-tune Llama 2 with an example dataset, using the Supervised Fine-Tuning (SFT) approach and Parameter-Efficient Fine-Tuning (PEFT) using LoRA.

We will use the [Guanaco dataset](#) from HuggingFace, which provides examples of 175 language tasks specifically designed for English grammar analysis, natural language understanding, cross-lingual self-awareness, and explicit content recognition. The dataset has 534,530 entries.

Here is the full script, which you can run in a Jupyter notebook, assuming it has access to a GPU and sufficient memory. Below we'll run through the code to explain

We use cookies on our site to give you the best experience possible. By continuing to browse the site, you agree to this use. For more information on how we use cookies, see our [Privacy Policy](#).

Accept

```

# Import necessary Libraries
import os
import torch
from datasets import load_dataset
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
from peft import LoraConfig
from trl import SFTTrainer
import torch
import gc

# Force garbage collection
gc.collect()

def display_cuda_memory():
    print("\n-----")
    print("torch.cuda.memory_allocated: %fGB"%(torch.cuda.memory_allocated()))
    print("torch.cuda.memory_reserved: %fGB"%(torch.cuda.memory_reserved()))
    print("torch.cuda.max_memory_reserved: %fGB"%(torch.cuda.max_memory_reserved()))
    print("\n-----\n")

# Install required libraries (uncomment the following line when running)

# For PyTorch memory management add the following code

os.environ["PYTORCH_CUDA_ALLOC_CONF"] = "max_split_size_mb:1024"

# Define model, dataset, and new model name
base_model = "NousResearch/Llama-2-7b-chat-hf"
guanaco_dataset = "mlabonne/guanaco-llama2-1k"
new_model = "llama-2-7b-chat-guanaco"

# Load dataset
dataset = load_dataset(guanaco_dataset, split="train")

# 4-bit Quantization Configuration
compute_dtype = getattr(torch, "float16")
quant_config = BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="fp4")

# Load model with 4-bit precision
model = AutoModelForCausalLM.from_pretrained(base_model, quantization_config=quant_config,
                                              model_config=dict(
                                                  use_cache=False,
                                                  pretraining_tp=1,
                                              ))

# Load tokenizer
tokenizer = AutoTokenizer.from_pretrained(base_model, trust_remote_code=True)
tokenizer.pad_token = tokenizer.eos_token
tokenizer.padding_side = "right"

# Set PEFT Parameters
peft_params = LoraConfig(lora_alpha=16, lora_dropout=0.1, r=64, bias="none",
                          target_modules=["q_proj", "v_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"],
                          inference_mode=False, lora_r=16)

```

We use cookies on our site to give you the best experience possible. By continuing to browse the site, you agree to this use. For more information on how we use cookies, see our [Privacy Policy](#).

Accept

./results", num_train_

```
# Initialize the trainer
trainer = SFTTrainer(model=model, train_dataset=dataset, peft_config=p

#Force clean the pytorch cache
gc.collect()

torch.cuda.empty_cache()

# Train the model
trainer.train()

# Save the model and tokenizer
trainer.model.save_pretrained(new_model)
trainer.tokenizer.save_pretrained(new_model)

# Evaluate the model (optional, requires Tensorboard installation)
# from tensorboard import notebook
# log_dir = "results/runs"
# notebook.start("--logdir {} --port 4000".format(log_dir))

# Test the model
logging.set_verbosity(logging.CRITICAL)
prompt = "Who is Leonardo Da Vinci?"
pipe = pipeline(task="text-generation", model=model, tokenizer=tokeniz
result = pipe(f"<s>[INST] {prompt} [/INST]")
print(result[0]['generated_text'])
```

Installing Libraries

The code below installs the required libraries. We will install the accelerate, peft, bitsandbytes, transformers, and trl. The transformers library provides access to pre-trained models and tokenizers, while bitsandbytes aids in efficient model quantization.

Note that if you are not using a Jupyter notebook, you'll need to run this outside the script.

```
%pip install accelerate==0.21.0 peft==0.4.0 bitsandbytes==0.40.2 trans
```

Importing Modules

We'll import required classes and functions. In particular, torch is the core library for PyTorch, a machine learning framework. load_dataset loads the training data. AutoModelForCausalLM and AutoTokenizer from transformers are used for loading the model and tokenizer, respectively. Others like BitsAndBytesConfig, TrainingArguments, pipeline, and logging provide configuration and utility functions.

We use cookies on our site to give you the best experience possible. By continuing to browse the site, you agree to this use. For more information on how we use cookies, see our [Privacy Policy](#).

Accept

```
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
from peft import LoraConfig
from trl import SFTTrainer
```

Model Configuration

Now, we'll define the base model for fine-tuning and the dataset to use. We'll set variables for the base model (NousResearch/Llama-2-7b-chat-hf), the dataset (mlabonne/guanaco-llama2-1k), and provide a name for the new model.

```
base_model = "NousResearch/Llama-2-7b-chat-hf"
guanaco_dataset = "mlabonne/guanaco-llama2-1k"
new_model = "llama-2-7b-chat-guanaco"
```

Loading Dataset

Next, we'll fetch and prepare the dataset for training. The `load_dataset` function retrieves the specified dataset from Hugging Face. Here, the instruction `split="train"` indicates we are using the training part of the dataset.

```
dataset = load_dataset(guanaco_dataset, split="train")
```

4-bit Quantization Configuration

We now need to configure the model for efficient training on consumer-grade hardware. This step sets up 4-bit quantization for the model using `BitsAndBytesConfig`. It's a way to reduce the model's memory footprint and computational requirements without significantly sacrificing performance.

```
compute_dtype = getattr(torch, "float16")
quant_config = BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_ty
```

Loading Model

The next step is to initialize the base model with the specified quantization settings. The `AutoModelForCausalLM.from_pretrained` function loads a pre-trained causal language model. It's configured to use the 4-bit quantization settings defined earlier. The `use_cache` and `pretraining_tp` settings optimize the model's training behavior for improved performance.

```
model = AutoModelForCausalLM.from_pretrained(base_model, quantization_
model.config.use_cache = False
model.config.pretraining_tp = 1
```

Now, we'll prepare the tokenizer to process text from the training dataset, in line with the model's requirements. The tokenizer converts text into a format that the model can understand. Setting `padding_side` to "right" addresses specific issues with fp16 (16-bit floating-point) operations.

```
tokenizer = AutoTokenizer.from_pretrained(base_model, trust_remote_cod
tokenizer.pad_token = tokenizer.eos_token
tokenizer.padding_side = "right"
```

Set PEFT Parameters

We'll now configure fine-tuning by updating a small subset of the model's parameters, using the LoRA (Low-Rank Adaptation) method. The `LoraConfig` class specifies settings for Parameter-Efficient Fine-Tuning (PEFT). Parameters like `lora_alpha`, `lora_dropout`, `r`, and `bias` define the architecture and behavior of the LoRA layers used for efficient fine-tuning. The `task_type` is set to "CAUSAL_LM" since LLaMA 2 is a causal language model.

```
peft_params = LoraConfig(lora_alpha=16, lora_dropout=0.1, r=64, bias="
```

Training Parameters

The next step is to define settings that control the training process. `TrainingArguments` sets up important training parameters like batch sizes, learning rate, weight decay, and others. Each parameter, such as `num_train_epochs` or `learning_rate`, controls a specific aspect of the training, like the number of epochs the model will train for or the initial learning rate for the optimizer.

```
training_params = TrainingArguments(output_dir="./results", num_train_
```

Model Fine-Tuning

Finally, we can start the actual fine-tuning process of the model with the dataset.

dataset.

```
trainer = SFTTrainer(model=model, train_dataset=dataset, peft_config=p
```

Training Execution

To execute the training process, we'll run the `train()` method of `SFTTrainer`. It adjusts the model's weights based on the input data and training parameters.



Platform

Resources

Company

[Book a Demo](#)

RELATED ARTICLES

- AI Developers
- Generative AI Engineer
- LLaMA 2 Fine Tuning
- LLM Tuning with LoRA
- Stable Diffusion Training
- TensorRT-LLM
- Transformer Model

We use cookies on our site to give you the best experience possible. By continuing to browse the site, you agree to this use. For more information on how we use cookies, see our [Privacy Policy](#).

Accept


```
trainer.train()
```

The output should look something like this:

[illegible]

Save and Evaluate

Now that training has run, we need to save the fine-tuned model and evaluate its performance.

We'll use Tensorboard to visualize training metrics, aiding in evaluating the model's performance.

```
trainer.model.save_pretrained(new_model)
trainer.tokenizer.save_pretrained(new_model)

from tensorboard import notebook
log_dir = "results/runs"
notebook.start("--logdir {} --port 4000".format(log_dir))
```

Test the Model

We can now test the fine-tuned model's capabilities, with a simple prompt to generate text. This is done using the pipeline function, which is a high-level utility for text generation. The output reflects how well the model has adapted to the new data.

```
logging.set_verbosity(logging.CRITICAL)
prompt = "Who is Leonardo Da Vinci?"
pipe = pipeline(task="text-generation", model=model, tokenizer=tokenizer)
result = pipe(f"<s>[INST] {prompt} [/INST]")
print(result[0]['generated text'])
```

We use cookies on our site to give you the best experience possible. By continuing to browse the site, you agree to this use. For more information on how we use cookies, see our [Privacy Policy](#).

Accept

RATE THIS GUIDE

1 Rating

Optimizing Your AI Infrastructure with Run:ai

Run:ai automates resource management and orchestration and reduces cost for the infrastructure used to train LLMs and other computationally intensive models. With Run:ai, you can automatically run as many compute intensive experiments as needed.

Here are some of the capabilities you gain when using Run:ai:

- **Advanced visibility**—create an efficient pipeline of resource sharing by pooling GPU compute resources.
- **No more bottlenecks**—you can set up guaranteed quotas of GPU resources, to avoid bottlenecks and optimize billing.
- **A higher level of control**—Run:ai enables you to dynamically change resource allocation, ensuring each job gets the resources it needs at any given time.

Run:ai simplifies machine learning infrastructure pipelines, helping data scientists accelerate their productivity and the quality of their models.

Learn more about the [Run:ai GPU virtualization platform](#).



PLATFORM

- Run:ai Cluster Engine
- Run:ai Control Plane
- Run:ai Dev

RESOURCES

- Blog
- Guides
- White Papers
- Case Studies
- Documentation

ABOUT

- About Us
- Our Partners
- Join Us

New York, NY, USA
Tel Aviv, Israel

